UNIVERSITÄT PASSAU

Software Analysis

Syntax-based Analysis

Gordon Fraser
Lehrstuhl für Software Engineering II

CCLearner

- Li, L., Feng, H., Zhuang, W., Meng, N., & Ryder, B. (2017, September). CClearner: A deep learning-based clone detection approach. In 2017 IEEE International Conference on Software Maintenance and Evolution (ICSME) (pp. 249-260). IEEE.

CCLearner

Table 4: Recall comparison among tools (%)

Tool	T1	T2	VST3	ST3	MT3	WT3/4	Average (R_{T1-ST3})
CCLearner	100	98	98	89	28	1	93
SourcererCC	100	97	92	67	5	0	80
NiCad	100	85	98	77	0	0	85
Deckard	96	82	78	78	69	53	83

Table 5: Comparison among tools for the sampled precision, and the number of reported and true clone pairs

Tool	$P_{estimated}$(%)	# of reported clone pairs	# of estimated true clone pairs
CCLearner	93	548,987	510,558
SourcererCC	98	265,611	260,299
NiCad	68	646,058	439,319
Deckard	71	2,301,526	1,634,083

CCLearner

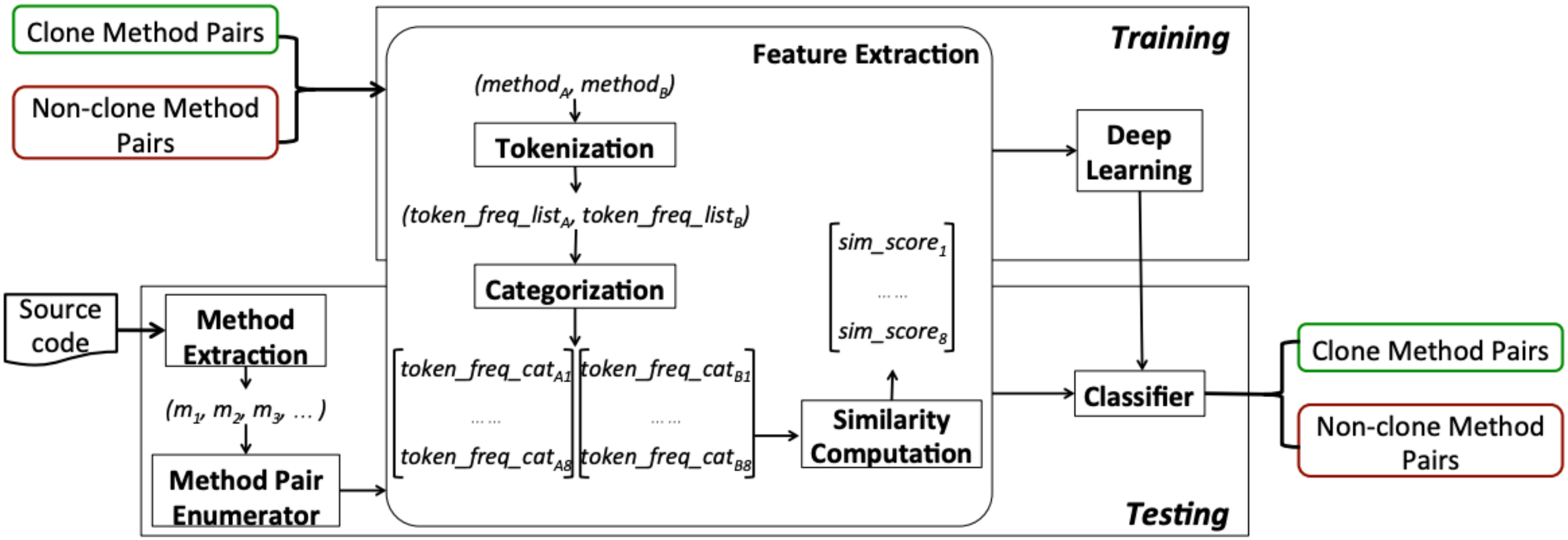
Index	Category name	Examples
C1	Reserved words	<if, 2>, <new, 3>, <try, 2>, …
C2	Operators	<+=, 2>, <!=, 3>, …
C3	Markers	<;, 2>, <[, 2>, <], 2>, …
C4	Literals	<1.3, 2>, <false, 3>, <null, 5>, …
C5	Type identifiers	<byte, 2>, <URLConnection, 1>, …
C6	Method identifiers	<read, 2>, <openConnection, 1>, …
C7	Qualified names	<System.out, 6>, <arr.length, 1>, …
C8	Variable identifiers	<conn, 2>, <numRead, 4>, …

CCLearner

$$sim\_score_i = 1 - \frac{\sum_x |freq(L_{Ai}, x) - freq(L_{Bi}, x)|}{\sum_x (freq(L_{Ai}, x) + freq(L_{Bi}, x))},$$
$$where\ x \in tokens(L_{Ai}) \cup tokens(L_{Bi}).$$

- Suppose we have two token-frequency lists:
  - LA = {< a, 3 >, < b, 4 >, < c, 5 >}
  - LB = {< b, 3 >, < c, 6 >, < d, 2 >}.
- The similarity score is computed as 1−(|3 − 0| + |4 − 3| + |5 − 6| + |0 − 2|) / ((3 + 0) + (4 + 3) + (5 + 6) + (0 + 2)) = 0.70.
- The more tokens shared between lists and the less frequency difference there is for each token, the higher similarity score
- if two methods have no token for a certain category (such as keywords), we set the corresponding similarity score as 0.5 by default

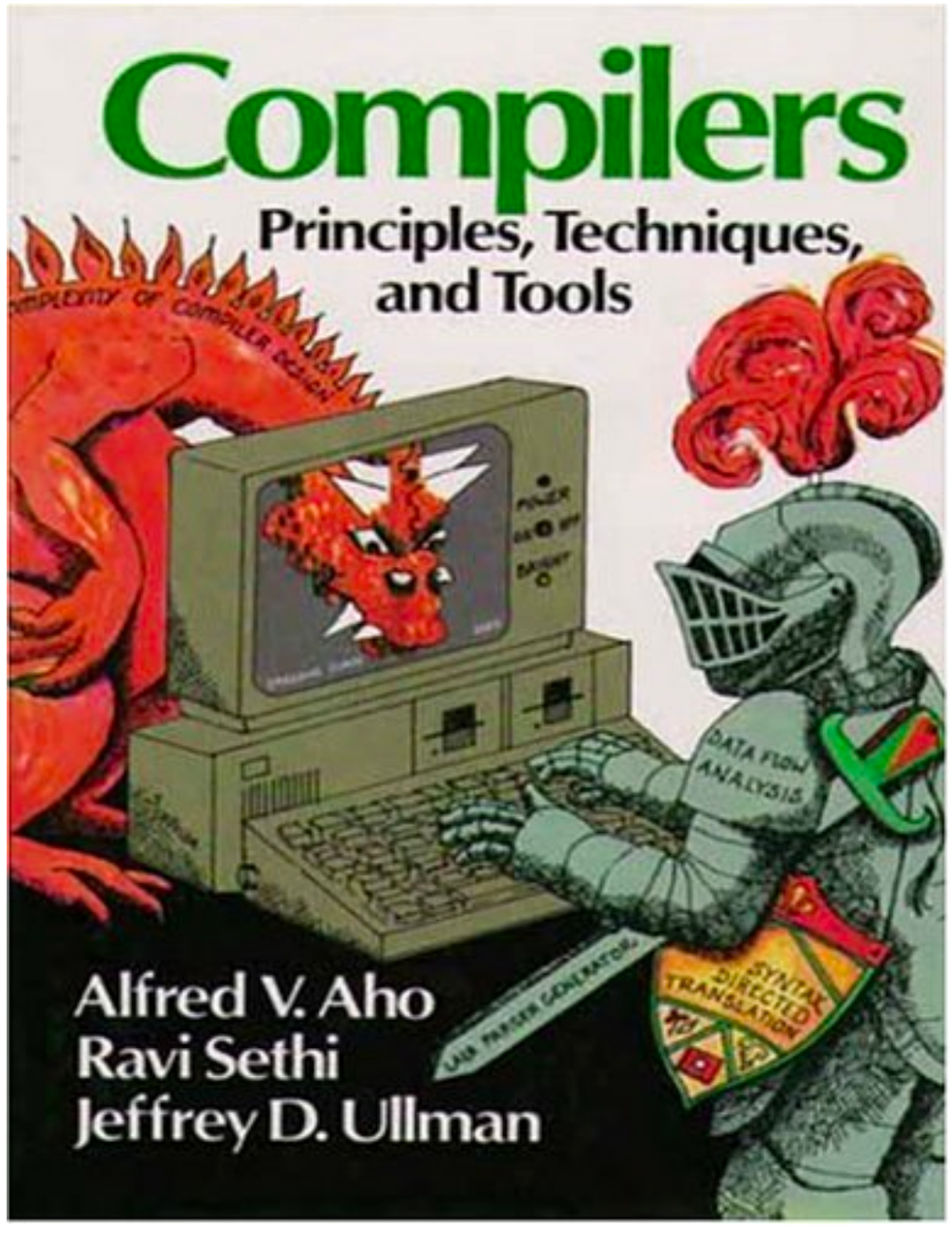
CCLearner


Clone Method Pairs
Non-clone Method Pairs
Training
Feature Extraction
(method_A, method_B)
Tokenization
(token_freq_list_A, token_freq_list_B)
Categorization
token_freq_cat_A1 ... token_freq_cat_A8
token_freq_cat_B1 ... token_freq_cat_B8
Similarity Computation
sim_score_1 ... sim_score_8
Deep Learning
Source code
Method Extraction
(m_1, m_2, m_3, ...)
Method Pair Enumerator
Classifier
Clone Method Pairs
Non-clone Method Pairs
Testing

CCLearner

Index	Category name	Examples
C1	Reserved words	<if, 2>, <new, 3>, <try, 2>, …
C2	Operators	<+=, 2>, <!=, 3>, …
C3	Markers	<;, 2>, <[, 2>, <], 2>, …
C4	Literals	<1.3, 2>, <false, 3>, <null, 5>, …
C5	Type identifiers	<byte, 2>, <URLConnection, 1>, …
C6	Method idenfiers	<read, 2>, <openConnection, 1>, …
C7	Qualified names	<System.out, 6>, <arr.length, 1>, …
C8	Variable identifiers	<conn, 2>, <numRead, 4>, …

How do we know what type of identifier a token is?



Chapters 1 and 2

Syntax Definition

- Context-free grammar is a 4-tuple with
  - A set of tokens (***terminal*** symbols)
  - A set of ***nonterminals***
  - A set of ***productions***
  - A designated ***start symbol***

Example Grammar

Context-free grammar for simple expressions:

$$G = <\{list, digit\}, \{\mathbf{+},\mathbf{-},\mathbf{0},\mathbf{1},\mathbf{2},\mathbf{3},\mathbf{4},\mathbf{5},\mathbf{6},\mathbf{7},\mathbf{8},\mathbf{9}\}, P, list>$$

with productions P =

$$list \rightarrow list \; \mathbf{+} \; digit$$

$$list \rightarrow list \; \mathbf{-} \; digit$$

$$list \rightarrow digit$$

$$digit \rightarrow \mathbf{0} \mid \mathbf{1} \mid \mathbf{2} \mid \mathbf{3} \mid \mathbf{4} \mid \mathbf{5} \mid \mathbf{6} \mid \mathbf{7} \mid \mathbf{8} \mid \mathbf{9}$$

Derivation

- Given a CF grammar we can determine the set of all *strings* (sequences of tokens) generated by the grammar using *derivation*
  - We begin with the start symbol
  - In each step, we replace one nonterminal in the current *sentential form* with one of the right-hand sides of a production for that nonterminal

$list \rightarrow list$ **+** $digit$

$list \rightarrow list$ **-** $digit$

$list \rightarrow digit$

$digit \rightarrow$ **0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9**

$\underline{list}$

$\Rightarrow \underline{list}$ **+** $digit$

$\Rightarrow \underline{list}$ **-** $digit$ **+** $digit$

$\Rightarrow \underline{digit}$ **-** $digit$ **+** $digit$

$\Rightarrow$ **9 -** $\underline{digit}$ **+** $digit$

$\Rightarrow$ **9 - 5 +** $\underline{digit}$

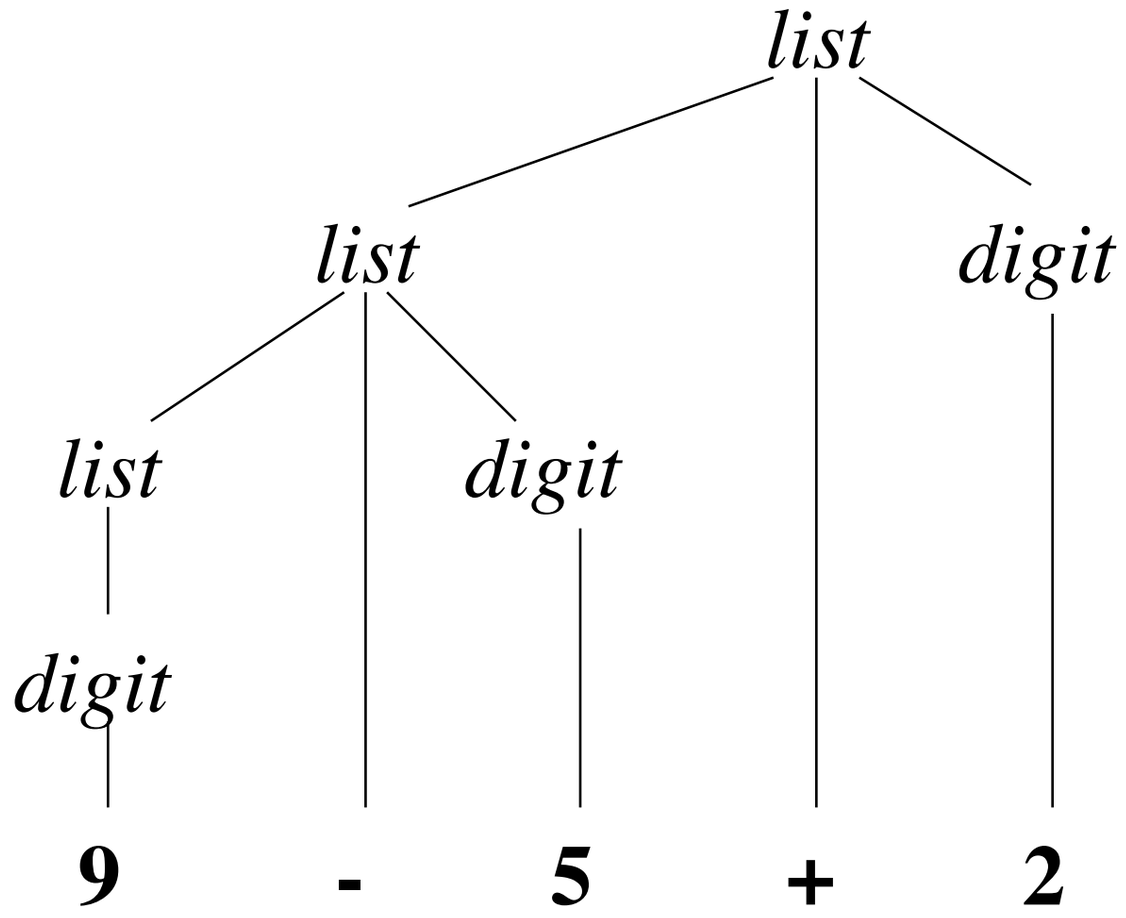
$\Rightarrow$ **9 - 5 + 2**

Parse Trees

- The ***root*** of the tree is labeled by the start symbol
- Each ***leaf*** of the tree is labeled by a terminal (=token) or ε
- Each ***interior node*** is labeled by a nonterminal
- If $A \rightarrow X_1\ X_2\ \ldots\ X_n$ is a production, then node A has immediate ***children*** $X_1, X_2, \ldots, X_n$ where X_i is a (non)terminal or ε (ε denotes the *empty string*)

Parse Tree for the Example Grammar

Parse tree of the string **9-5+2** using grammar *G*



list → *list* **+** *digit*

list → *list* **-** *digit*

list → *digit*

digit → **0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9**

Ambiguity

Consider the following context-free grammar:

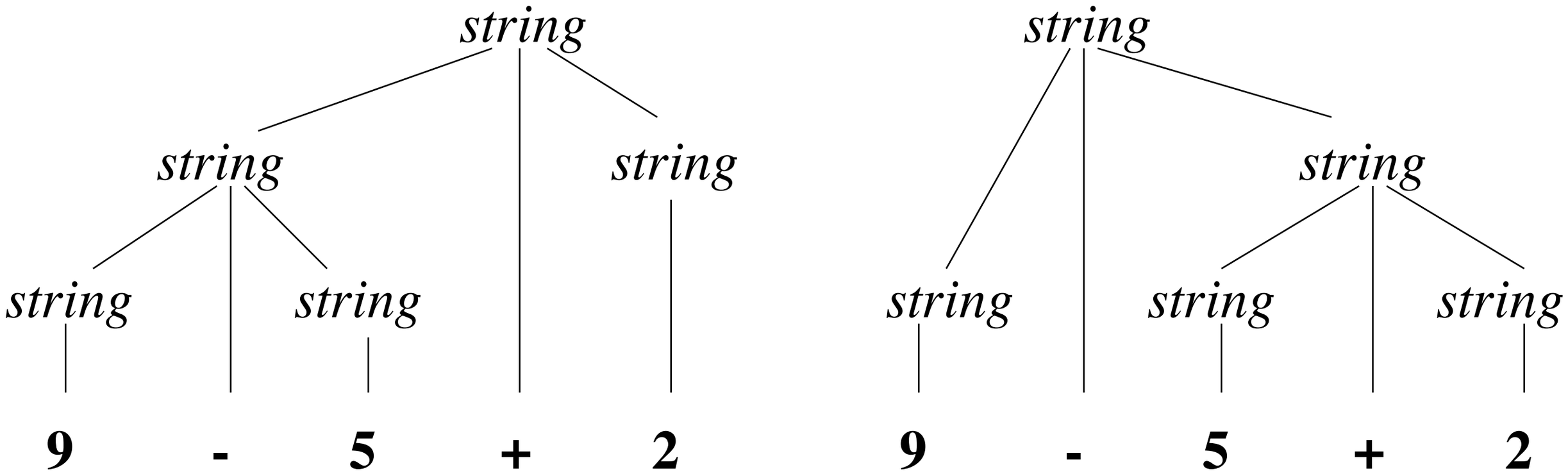
$$G = <\{string\}, \{\mathbf{+},\mathbf{-},\mathbf{0},\mathbf{1},\mathbf{2},\mathbf{3},\mathbf{4},\mathbf{5},\mathbf{6},\mathbf{7},\mathbf{8},\mathbf{9}\}, P, string>$$

with production P =

$$string \rightarrow string\ \mathbf{+}\ string \mid string\ \mathbf{-}\ string \mid \mathbf{0} \mid \mathbf{1} \mid \ldots \mid \mathbf{9}$$

This grammar is *ambiguous*, because more than one parse tree represents the string **9-5+2**

Ambiguity


string
string
string
string
string
string
string
9
-
5
+
2
string
string
string
string
string
string
string
9
-
5
+
2

Associativity of Operators

Left-associative operators have *left-recursive* productions

$$left \rightarrow left \; \mathbf{+} \; term \mid term$$

String **a+b+c** has the same meaning as **(a+b)+c**

Right-associative operators have *right-recursive* productions

$$right \rightarrow term \; \mathbf{=} \; right \mid term$$

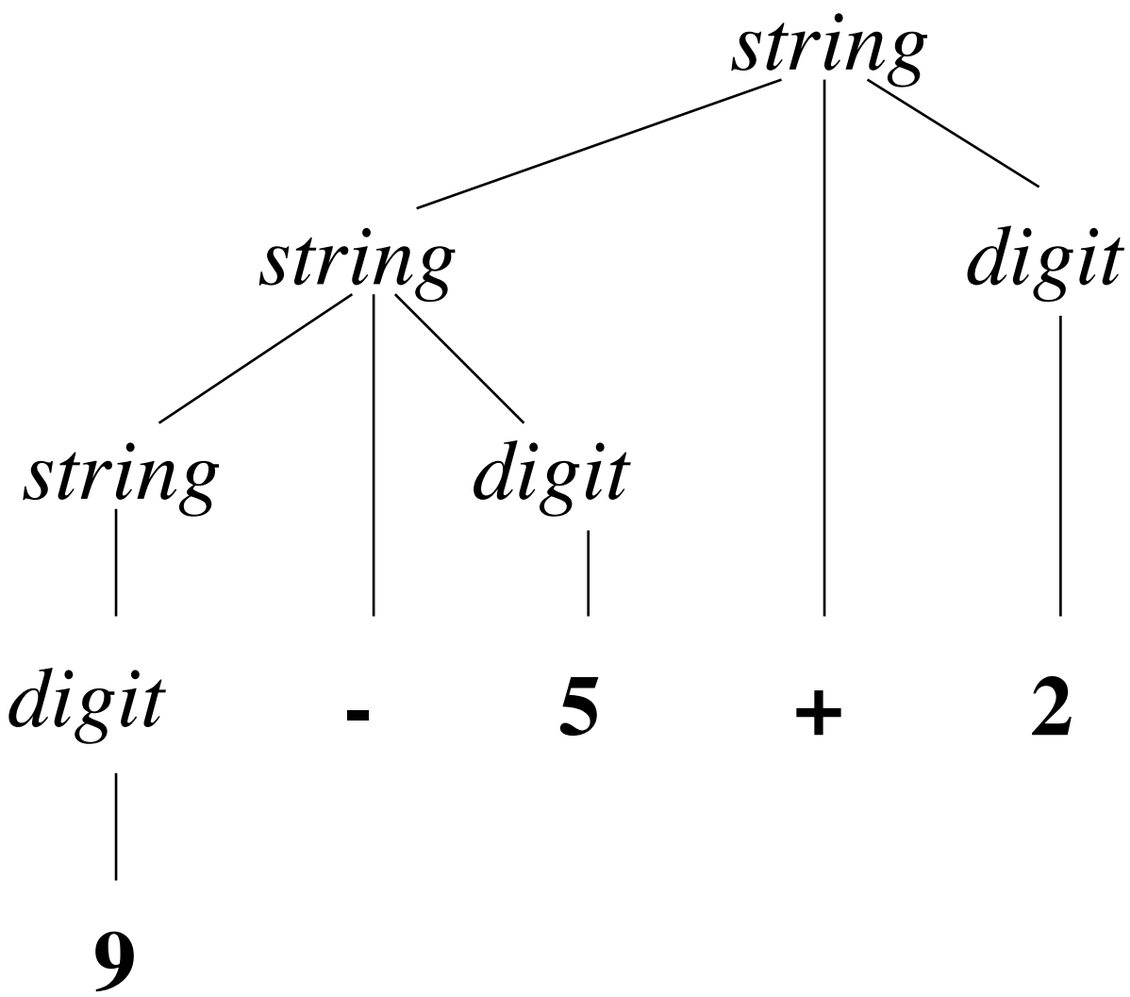
String **a=b=c** has the same meaning as **a=(b=c)**

Associativity of Operators

string → *string* **+** *string* | *string* **-** *string* | **0** | **1** | … | **9**

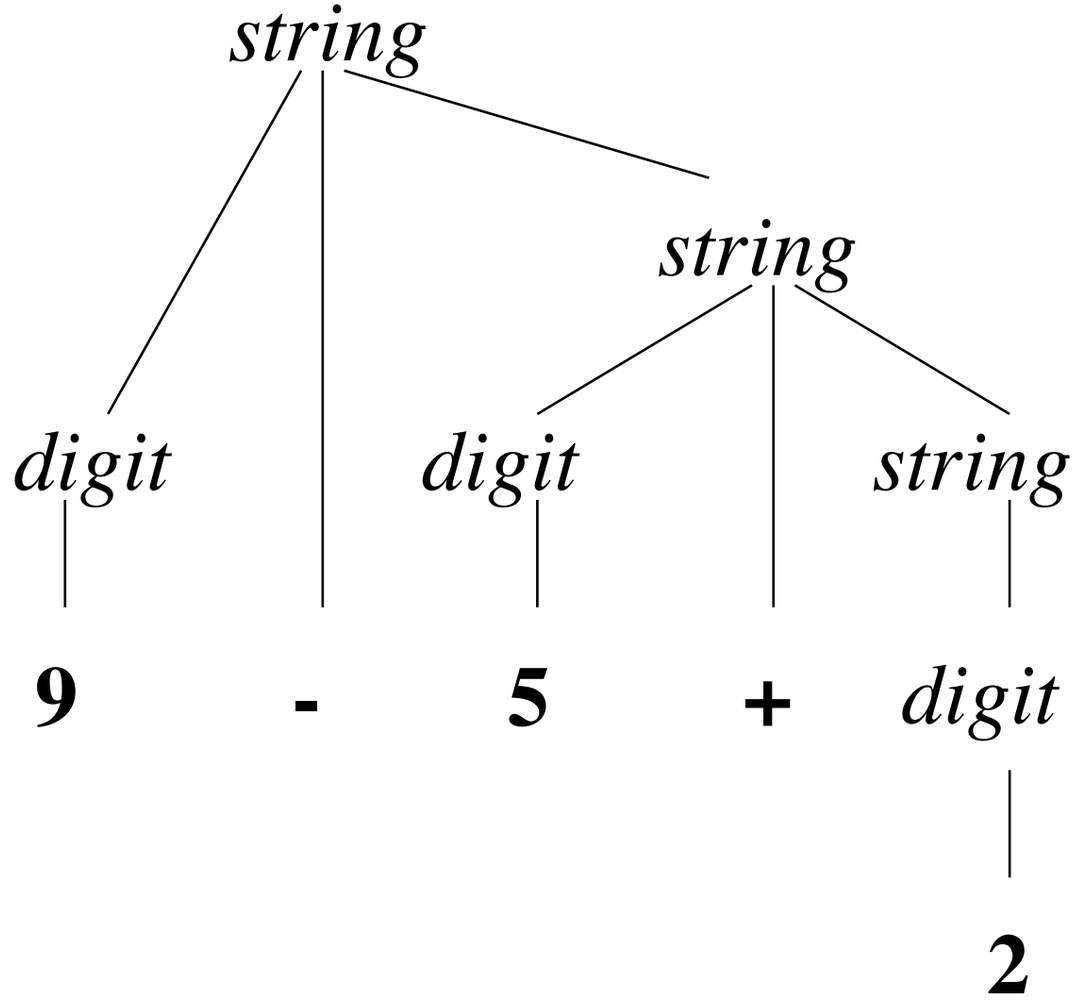
string → *string* **+** *digit*
| *string* **-** *digit* | *digit*

digit → **0** | **1** | … | **9**



string → *digit* **+** *string*
| *digit* **-** *string* | *digit*

digit → **0** | **1** | … | **9**

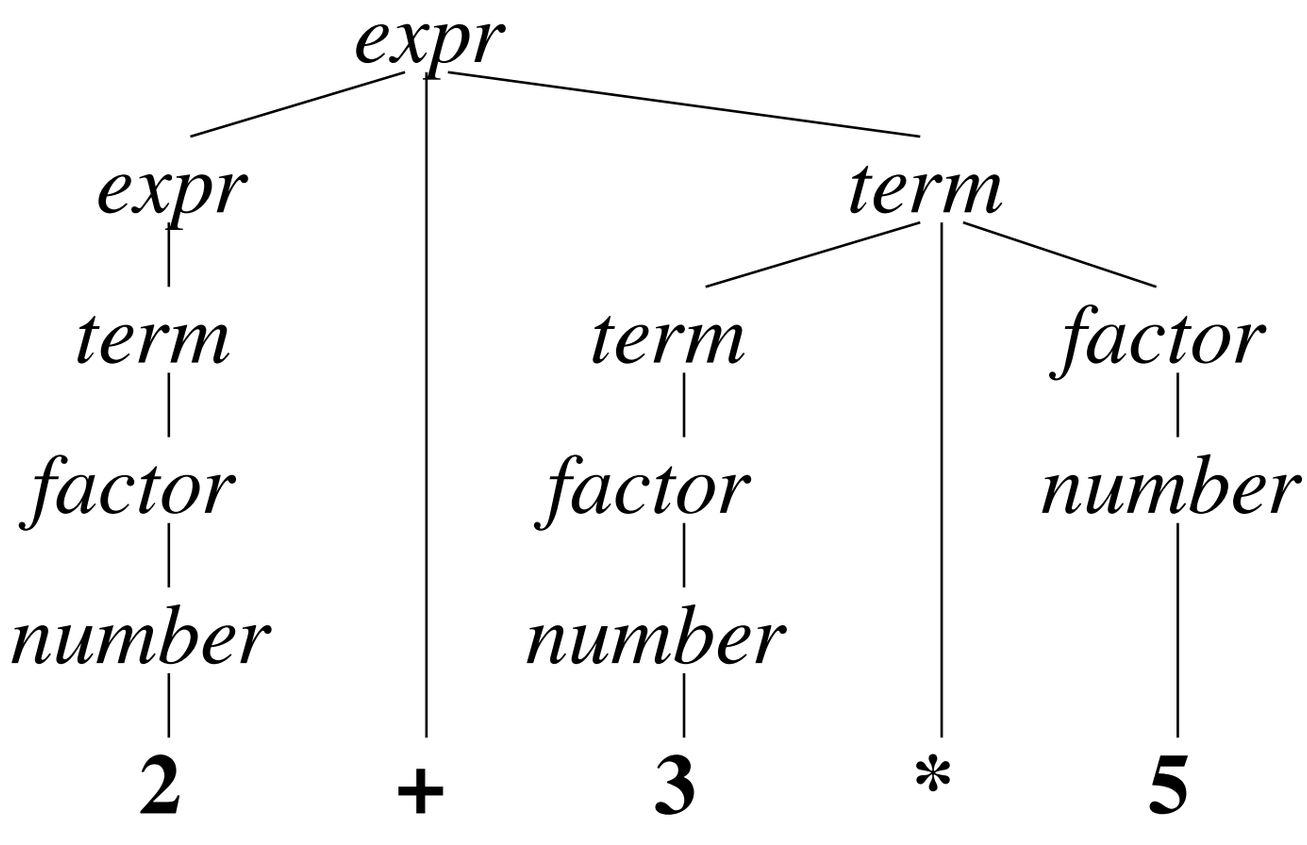


Precedence of Operators

Operators with higher precedence "bind more tightly"

$$expr \rightarrow expr + term \mid term$$
$$term \rightarrow term * factor \mid factor$$
$$factor \rightarrow number \mid (expr)$$

String **2+3*5** has the same meaning as **2+(3*5)**



Syntax of Statements

stmt → **id :=** *expr*
| **if** *expr* **then** *stmt*
| **if** *expr* **then** *stmt* **else** *stmt*
| **while** *expr* **do** *stmt*
| **begin** *opt_stmts* **end**

opt_stmts → *stmt* **;** *opt_stmts*
| ε

Parsing

- ***Parsing*** = *process of determining if a string of tokens can be generated by a grammar*
- For any CF grammar there is a parser that takes at most $O(n^3)$ time to parse a string of n tokens
- Linear algorithms suffice for parsing programming language source code
- ***Top-down parsing*** "constructs" a parse tree from root to leaves
- ***Bottom-up parsing*** "constructs" a parse tree from leaves to root

Predictive Parsing

- *Recursive descent parsing* is a top-down parsing method
  - Each nonterminal has one (recursive) procedure that is responsible for parsing the nonterminal's syntactic category of input tokens
  - When a nonterminal has multiple productions, each production is implemented in a branch of a selection statement based on input look-ahead information
- *Predictive parsing* is a special form of recursive descent parsing where we use one lookahead token to unambiguously determine the parse operations

Example Predictive Parser (Grammar)

type → *simple*
| **^ id**
| **array [** *simple* **] of** *type*
simple → **integer**
| **char**
| **num dotdot num**

Example Predictive Parser (Program Code)

```
procedure match(t : token);
begin
    if lookahead = t then
        lookahead := nexttoken()
    else error()
end;
```

```
procedure type();
begin
    if lookahead in { 'integer', 'char', 'num' } then
        simple()
    else if lookahead = '^' then
        match('^'); match(id)
    else if lookahead = 'array' then
        match('array'); match('['); simple();
        match(']'); match('of'); type()
    else error()
end;
```

```
procedure simple();
begin
    if lookahead = 'integer' then
        match('integer')
    else if lookahead = 'char' then
        match('char')
    else if lookahead = 'num' then
        match('num');
        match('dotdot');
        match('num')
    else error()
end;
```

Example Predictive Parser (Execution Step 1)

type()

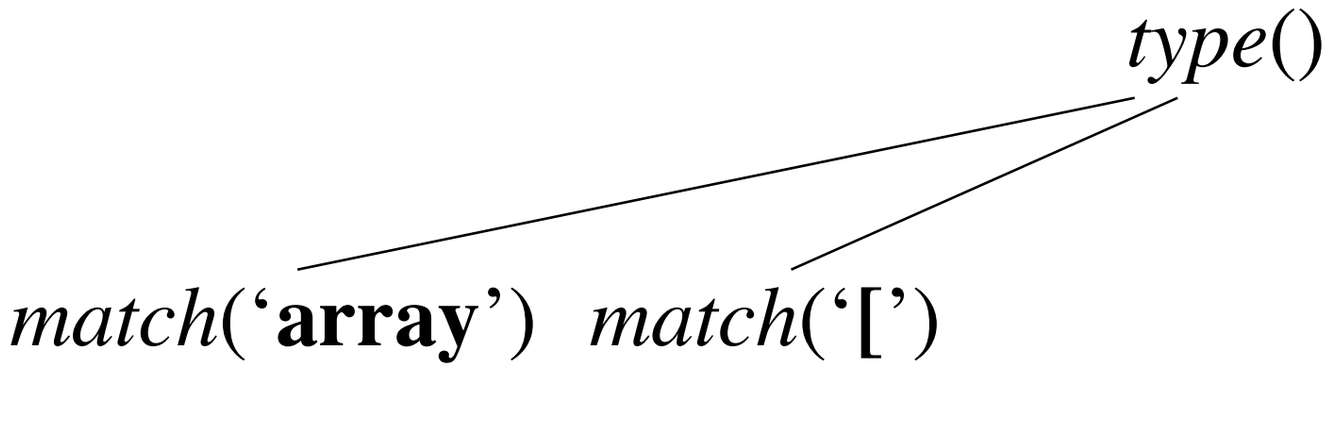
Check *lookahead* and call *match*

match(‘**array**’)

Input: **array** **[** **num** **dotdot** **num** **]** **of** **integer**

lookahead

Example Predictive Parser (Execution Step 2)



Input: **array [num dotdot num] of integer**

lookahead

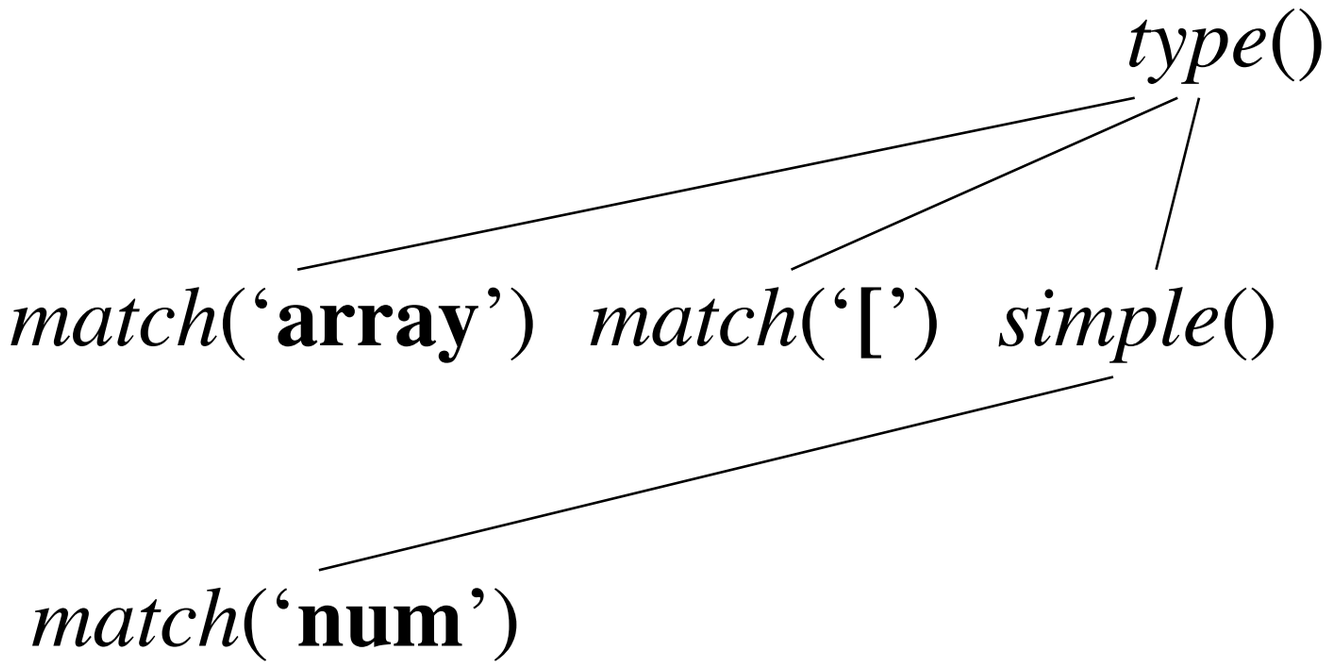
Example Predictive Parser (Program Code)

```
procedure match(t : token);
begin
    if lookahead = t then
        lookahead := nexttoken()
    else error()
end;
```

```
procedure type();
begin
    if lookahead in { 'integer', 'char', 'num' } then
        simple()
    else if lookahead = '^' then
        match('^'); match(id)
    else if lookahead = 'array' then
        match('array'); match('['); simple();
        match(']'); match('of'); type()
    else error()
end;
```

```
procedure simple();
begin
    if lookahead = 'integer' then
        match('integer')
    else if lookahead = 'char' then
        match('char')
    else if lookahead = 'num' then
        match('num');
        match('dotdot');
        match('num')
    else error()
end;
```

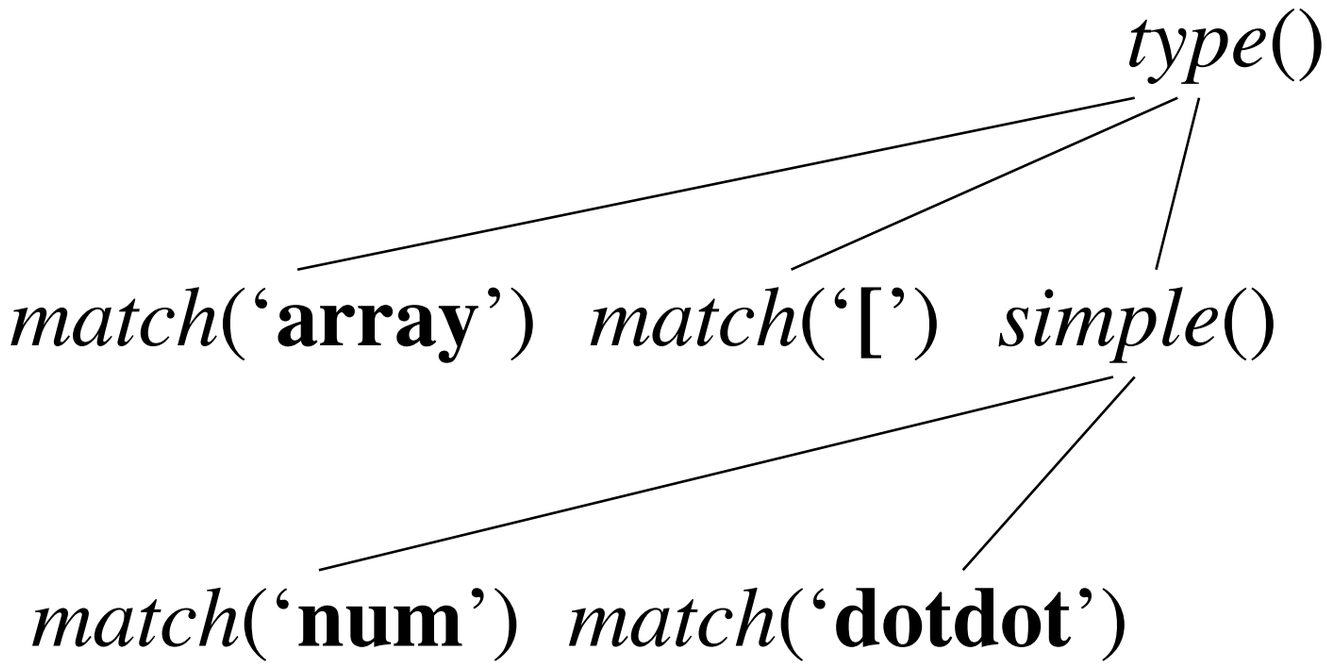
Example Predictive Parser (Execution Step 3)



Input: **array [num dotdot num] of integer**

lookahead

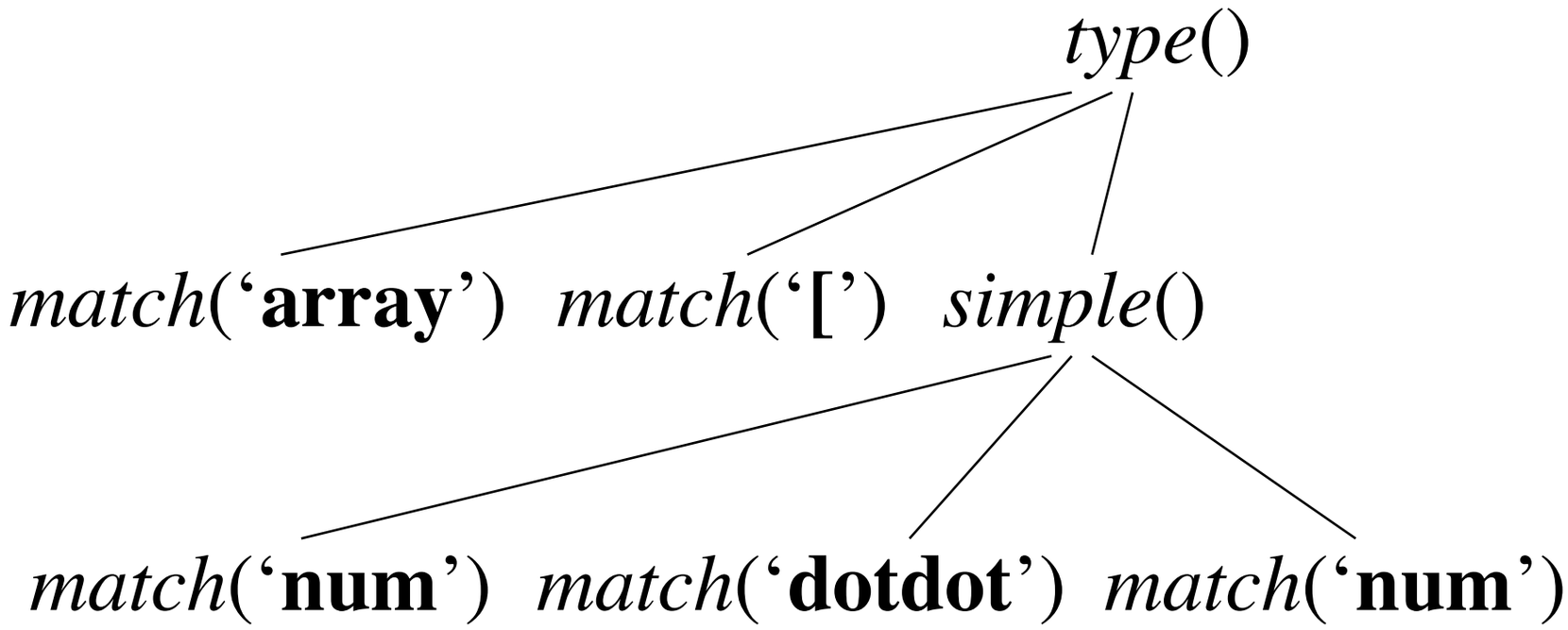
Example Predictive Parser (Execution Step 4)



Input: **array [num dotdot num] of integer**

lookahead

Example Predictive Parser (Execution Step 5)



Input: **array [num dotdot num] of integer**

lookahead

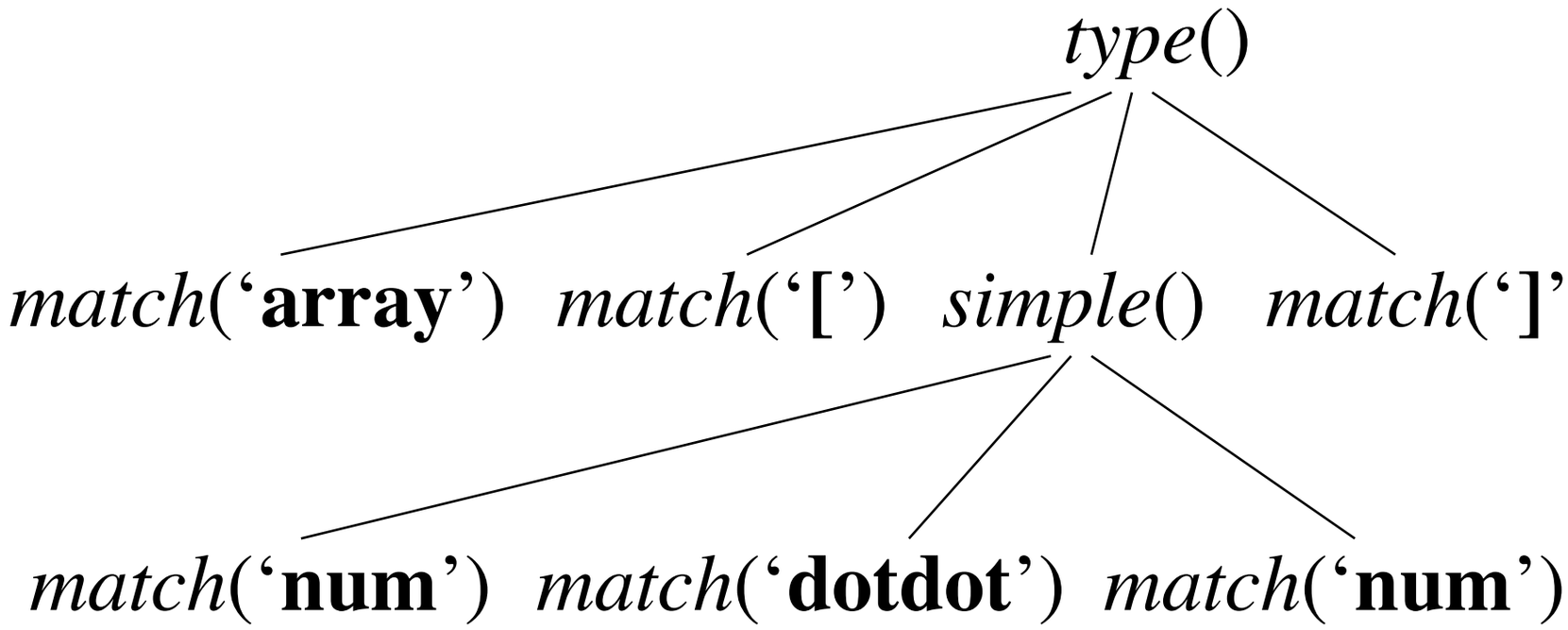
Example Predictive Parser (Program Code)

```
procedure match(t : token);
begin
    if lookahead = t then
        lookahead := nexttoken()
    else error()
end;
```

```
procedure type();
begin
    if lookahead in { 'integer', 'char', 'num' } then
        simple()
    else if lookahead = '^' then
        match('^'); match(id)
    else if lookahead = 'array' then
        match('array'); match('['); simple();
        match(']'); match('of'); type()
    else error()
end;
```

```
procedure simple();
begin
    if lookahead = 'integer' then
        match('integer')
    else if lookahead = 'char' then
        match('char')
    else if lookahead = 'num' then
        match('num');
        match('dotdot');
        match('num')
    else error()
end;
```

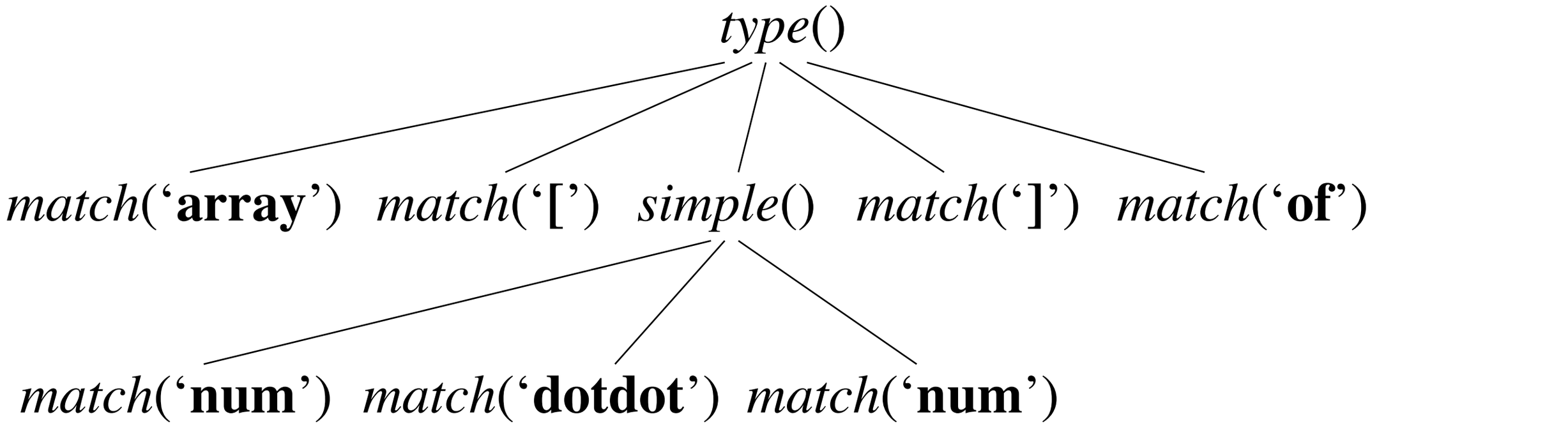
Example Predictive Parser (Execution Step 6)



Input: **array [num dotdot num] of integer**

lookahead

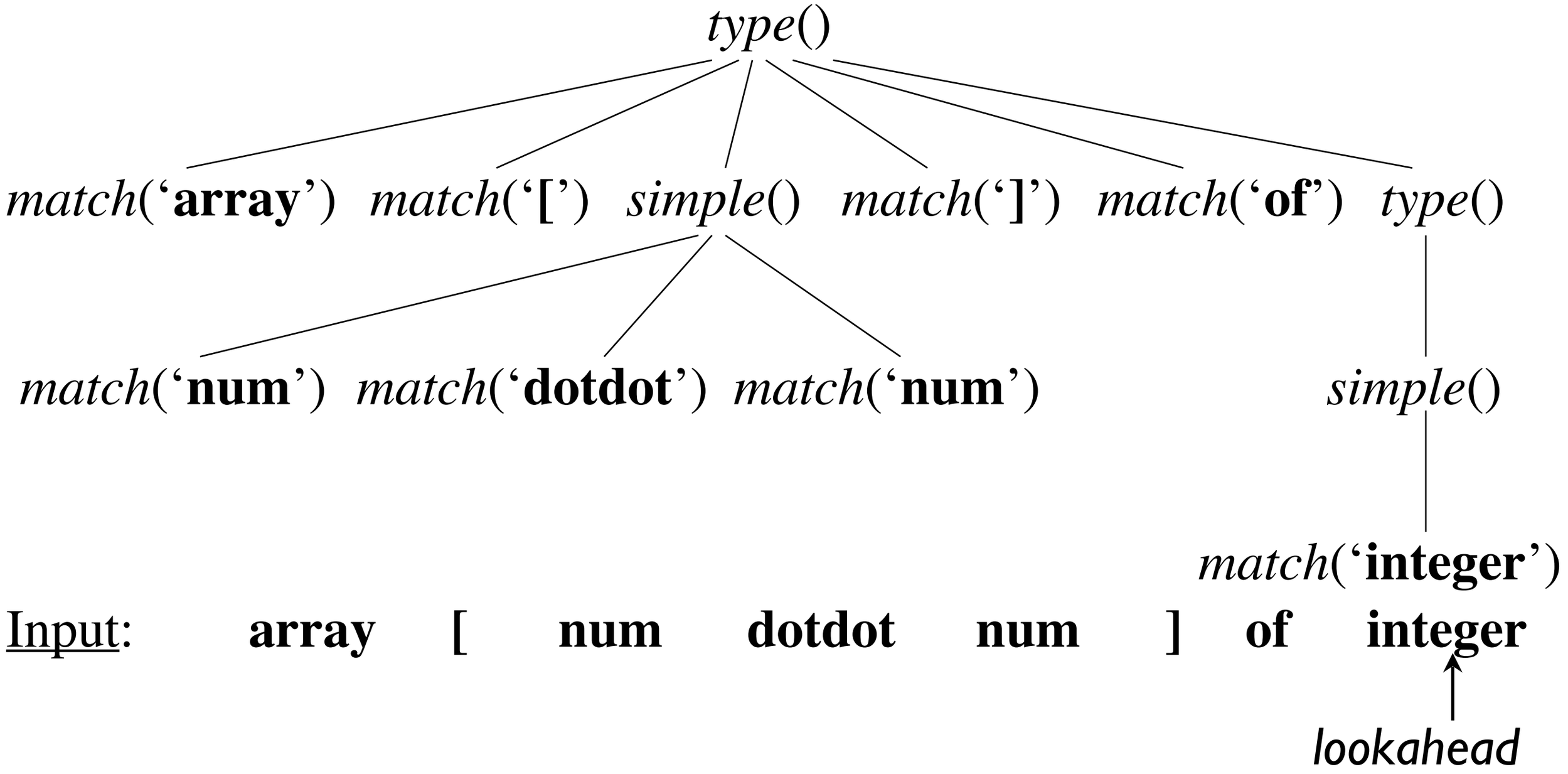
Example Predictive Parser (Execution Step 7)



Input: **array [num dotdot num] of integer**

lookahead

Example Predictive Parser (Execution Step 8)



FIRST

FIRST(α) is the set of terminals that appear as the first symbols of one or more strings generated from α

type → *simple*
| **^ id**
| **array [** *simple* **] of** *type*
simple → **integer**
| **char**
| **num dotdot num**

FIRST(*simple*) = { **integer**, **char**, **num** }
FIRST(**^ id**) = { **^** }
FIRST(*type*) = { **integer**, **char**, **num**, **^**, **array** }

How to use FIRST

We use FIRST to write a predictive parser as follows

expr → *term rest*

rest → **+** *term rest*

| **-** *term rest*

| ε

```
procedure rest();
begin
   if lookahead in FIRST(+ term rest) then
      match('+'); term(); rest()
   else if lookahead in FIRST(- term rest) then
      match('-'); term(); rest()
   else return
end;
```

When a nonterminal *A* has two (or more) productions as in

$$A \rightarrow \alpha$$
$$| \beta$$

Then FIRST (α) and FIRST(β) must be disjoint for predictive parsing to work

Left Factoring

When more than one production for nonterminal *A* starts with the same symbols, the FIRST sets are not disjoint

stmt → **if** *expr* **then** *stmt* **endif**
| **if** *expr* **then** *stmt* **else** *stmt* **endif**

We can use *left factoring* to fix the problem

stmt → **if** *expr* **then** *stmt opt_else*
opt_else → **else** *stmt* **endif**
| **endif**

Left Recursion

When a production for nonterminal *A* starts with a self reference then a predictive parser loops forever

$$A \rightarrow A\ \alpha$$
$$\mid \beta$$
$$\mid \gamma$$

We can eliminate ***left recursive productions*** by systematically rewriting the grammar using ***right recursive productions***

$$A \rightarrow \beta\ R$$
$$\mid \gamma\ R$$
$$R \rightarrow \alpha\ R$$
$$\mid \varepsilon$$

Example Grammar

expr → *expr* **+** *term*
expr → *expr* **-** *term*
expr → *term*
term → *term* * *factor*
term → *term* / *factor*
term → *factor*
factor → (*expr*)
factor → **ID**
factor → **NUMBER**

expr → *term expr'*
expr' → **+** *expr* | **-** *expr* | ε
term → *factor term'*
term' → * *term* | / *term* | ε
factor → (*expr*)
factor → **ID**
factor → **NUMBER**

Syntax-Directed Translation

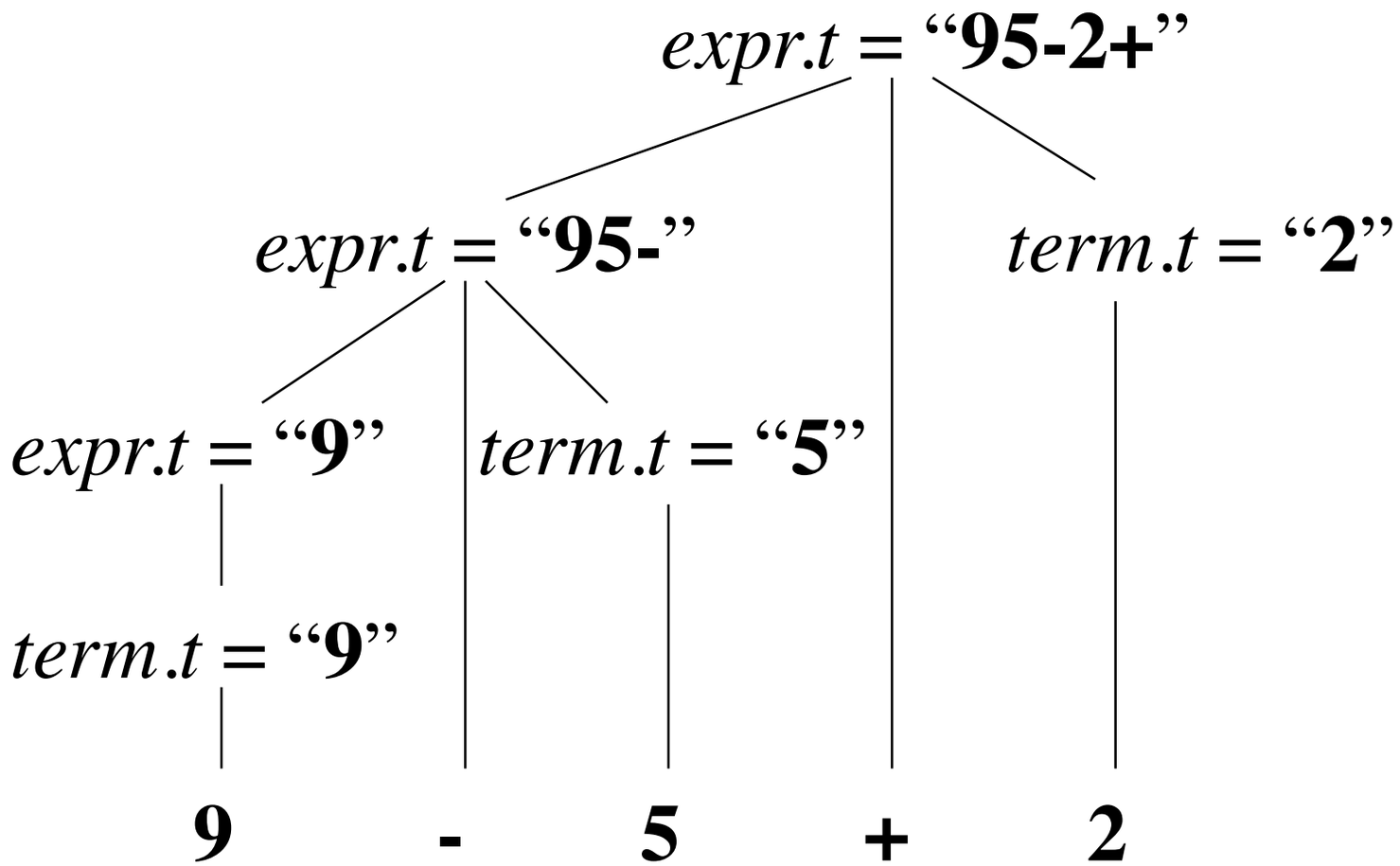
- Uses a CF grammar to specify the syntactic structure of the language
- AND associates a set of ***attributes*** with the terminals and nonterminals of the grammar
- AND associates with each production a set of ***semantic rules*** to compute values of attributes
- A parse tree is traversed and semantic rules applied: after the tree traversal(s) are completed, the attribute values on the nonterminals contain the translated form of the input

Example Attribute Grammar

(String concat operator)

Production	Semantic Rule
$expr \rightarrow expr_1$ **+** $term$	$expr.t := expr_1.t \,//\, term.t \,//$ "+"
$expr \rightarrow expr_1$ **-** $term$	$expr.t := expr_1.t \,//\, term.t \,//$ "-"
$expr \rightarrow term$	$expr.t := term.t$
$term \rightarrow$ **0**	$term.t :=$ "0"
$term \rightarrow$ **1**	$term.t :=$ "1"
…	…
$term \rightarrow$ **9**	$term.t :=$ "9"

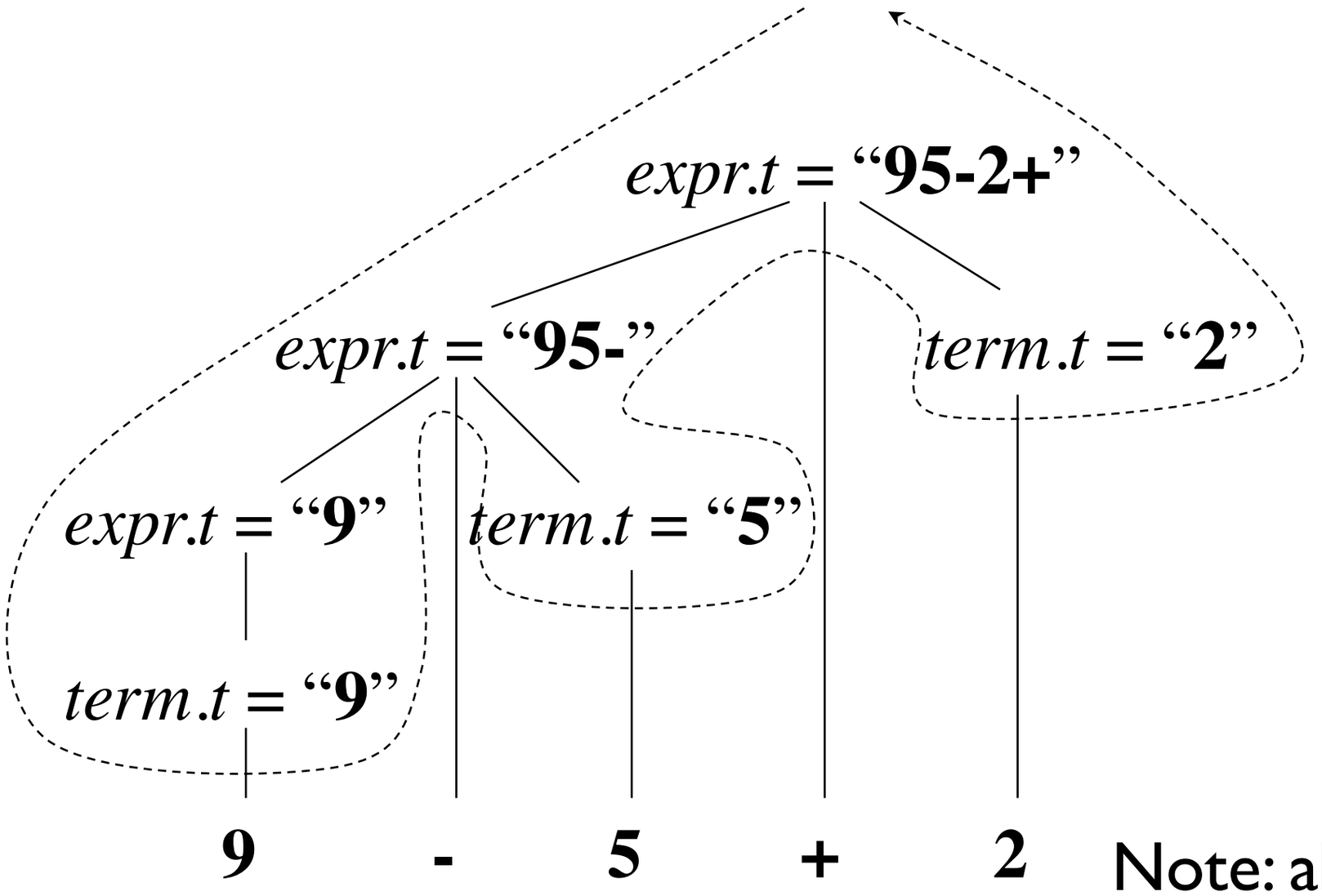
Example Annotated Parse Tree



Depth-First Traversal

```
procedure visit(n : node);
begin
    for each child m of n, from left to right do
        visit(m);
    evaluate semantic rules at node n
end
```

Depth-First Traversal (Example)



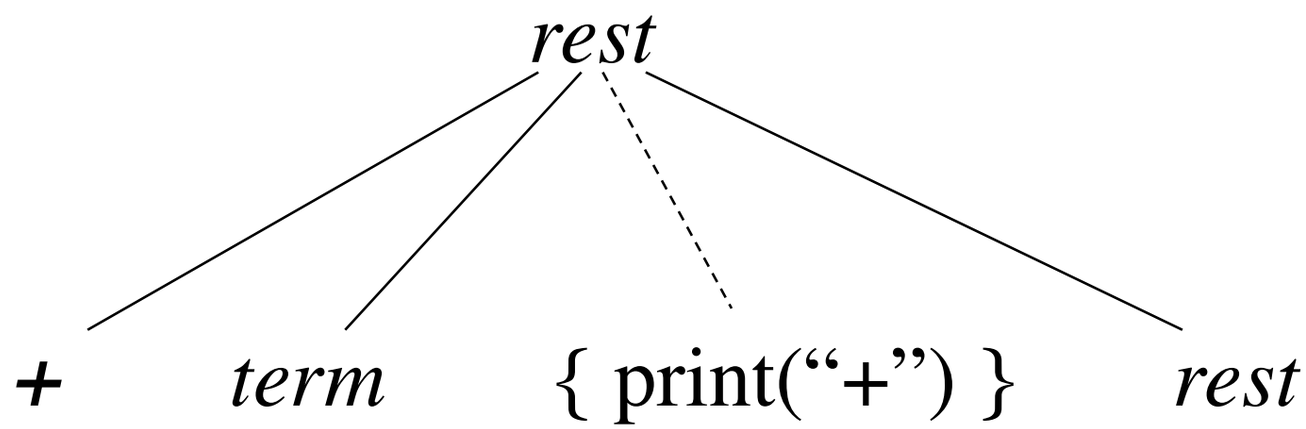
Note: all attributes are of the synthesised type

Translation Schemes

- A ***translation scheme*** is a CF grammar embedded with *semantic actions*

$$rest \rightarrow \mathbf{+}\ term\ \{\ \text{print}(\text{“+”})\ \}\ rest$$

Embedded
semantic action



Example Translation Scheme

expr → *expr* **+** *term* { print(“+”) }

expr → *expr* **-** *term* { print(“-”) }

expr → *term*

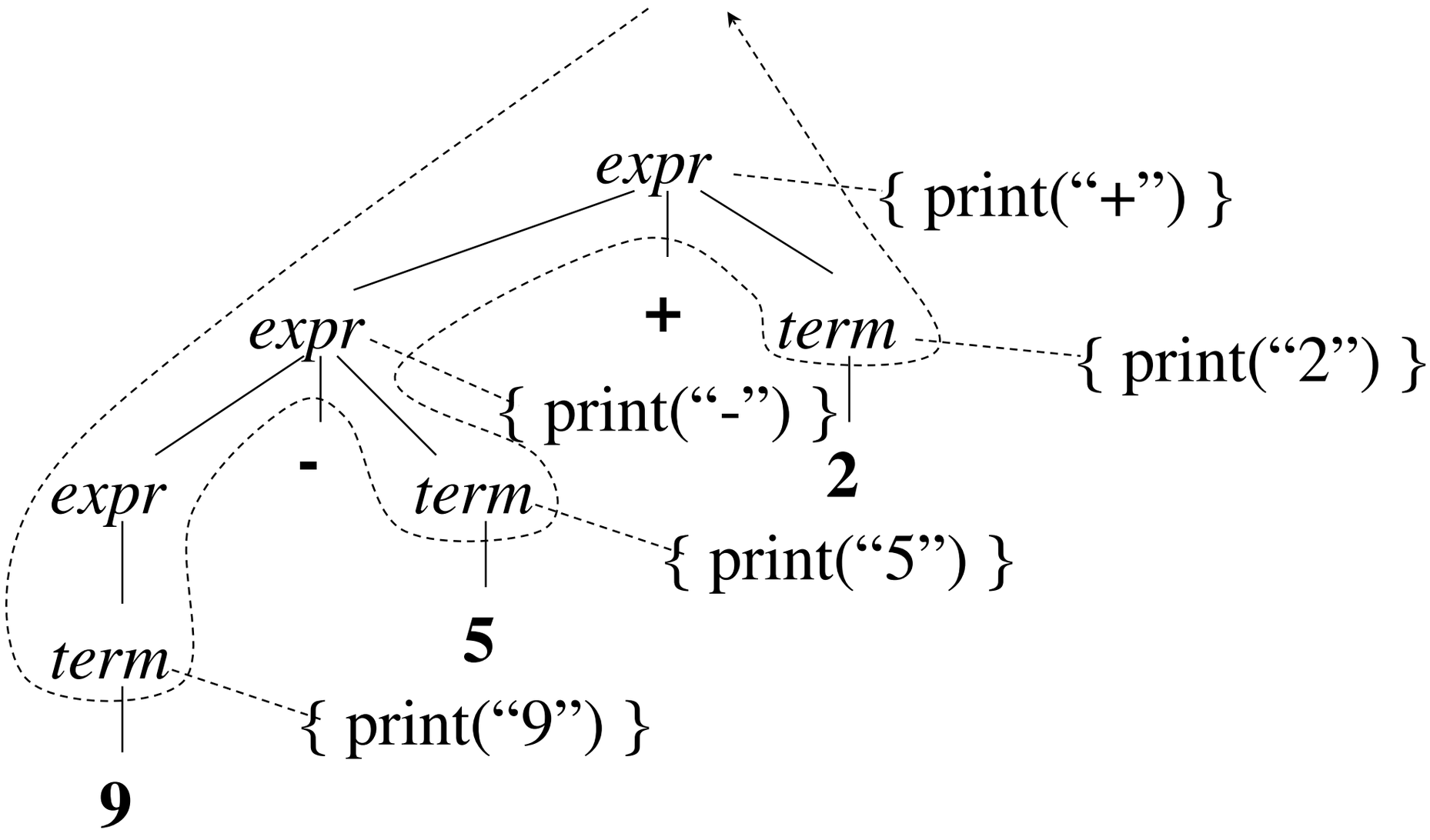
term → **0** { print(“0”) }

term → **1** { print(“1”) }

… …

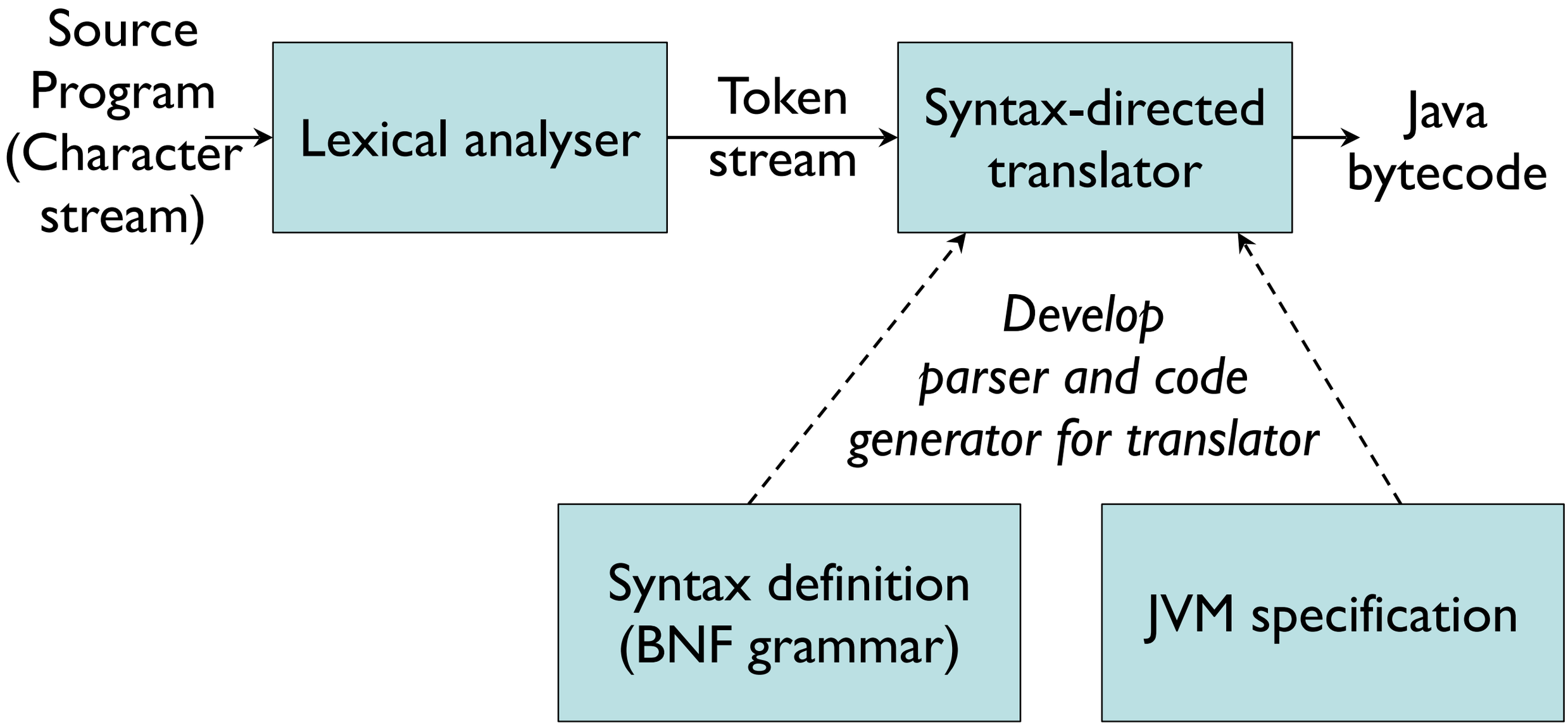
term → **9** { print(“9”) }

Example Translation Scheme

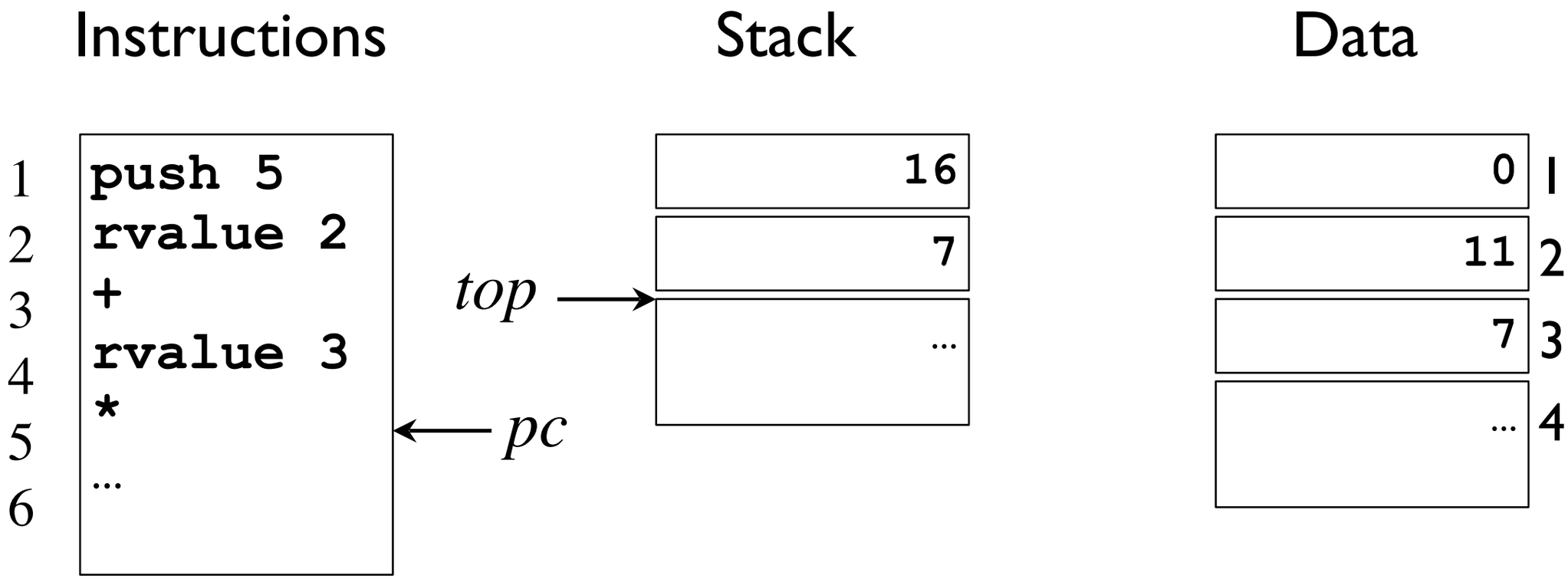


Translates **9-5+2** into postfix **95-2+**

A Simple Java Compiler



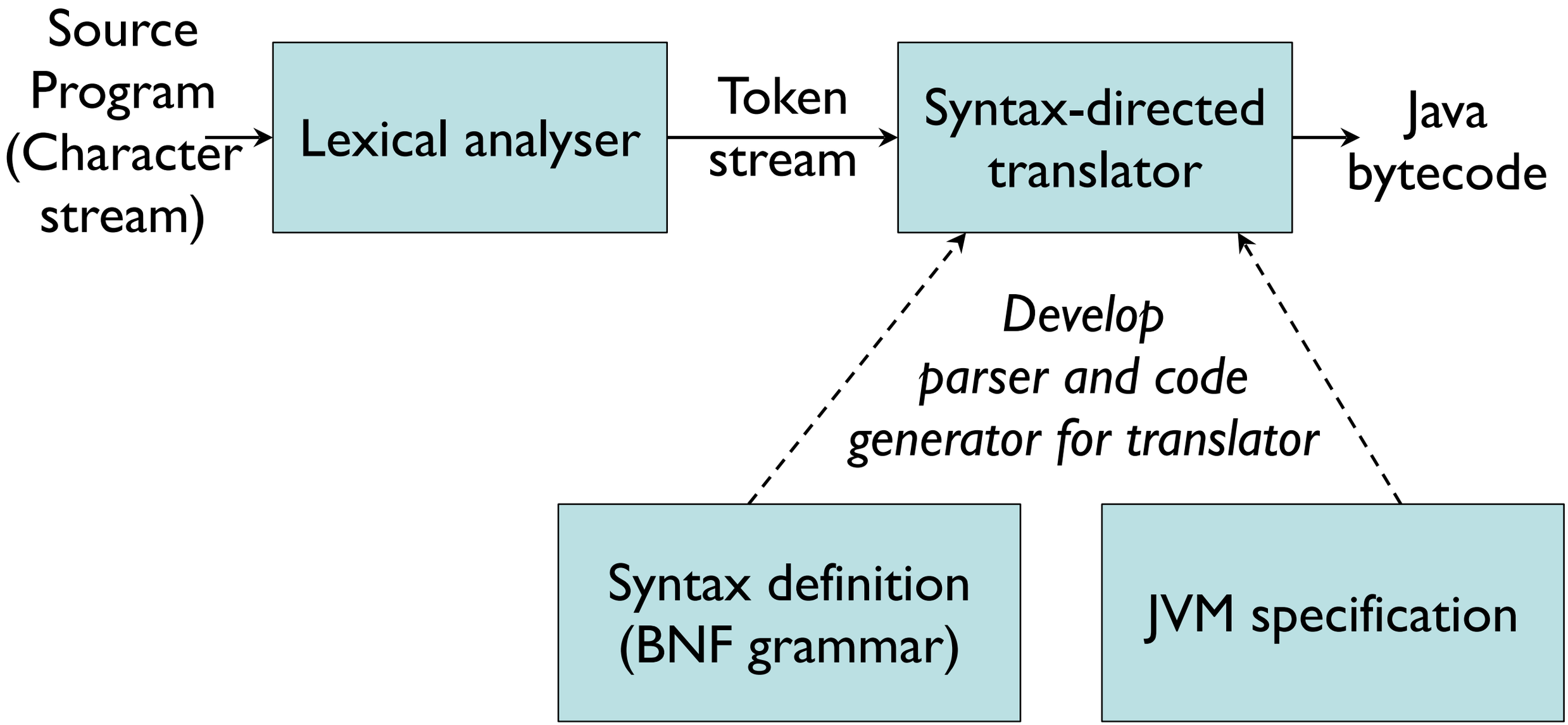
Abstract Stack Machines



Generic Instructions for Stack Manipulation

PUSH *v*	push constant value *v* onto the stack
RVALUE *l*	push contents of data location *l*
LVALUE *l*	push address of data location *l*
IADD	add value on top with value below it pop both and push result
ISUB	subtract value on top from value below it pop both and push result
***, /,** ...	ditto for other arithmetic operations
<, &, ...	ditto for relational and logical operations
LABEL *l*	label instruction with *l*
GOTO *l*	jump to instruction labeled *l*
GOFALSE *l*	pop the top value, if zero then jump to *l*
GOTRUE *l*	pop the top value, if nonzero then jump to *l*
HALT	stop execution

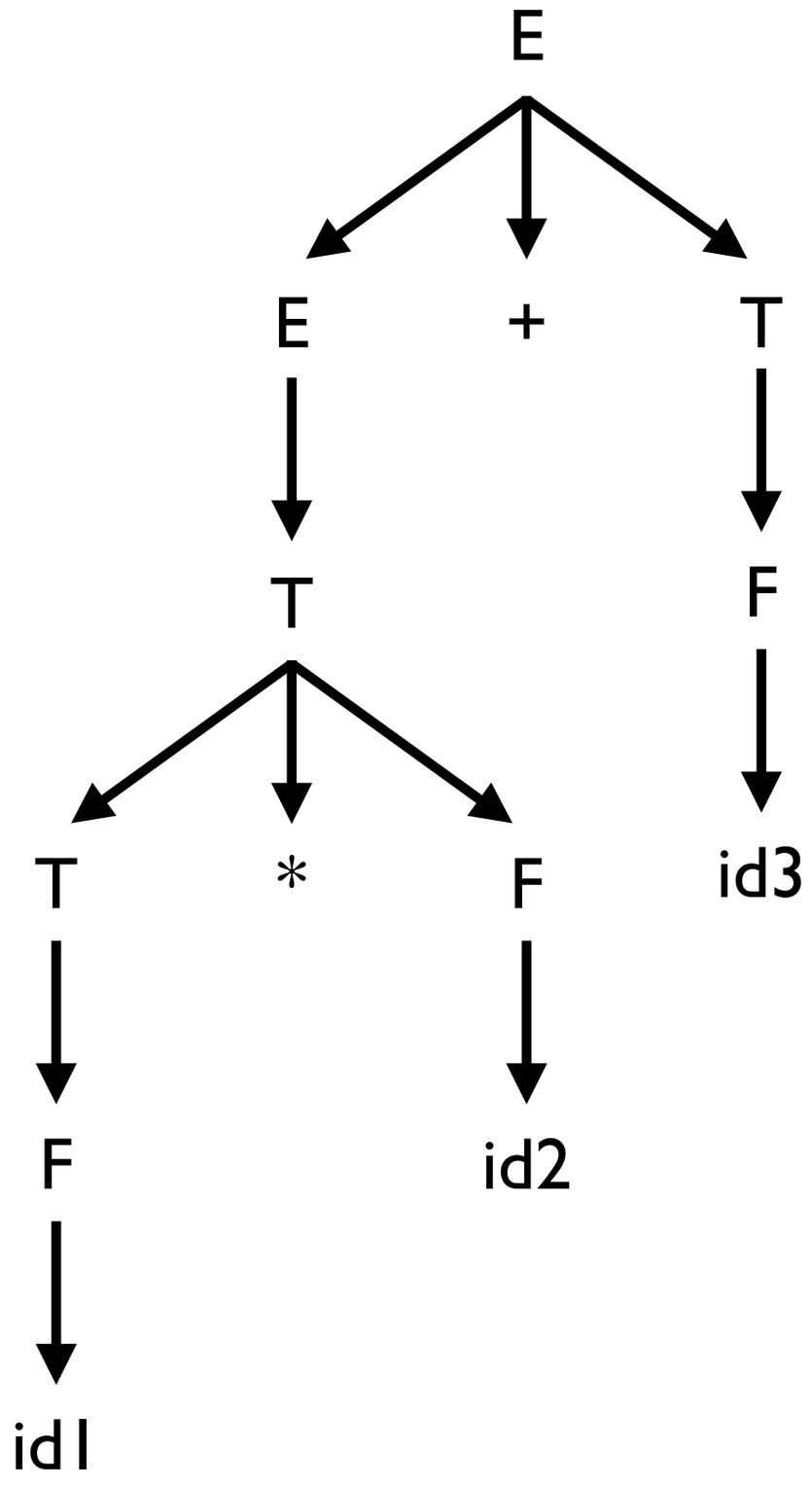
A Simple Java Compiler

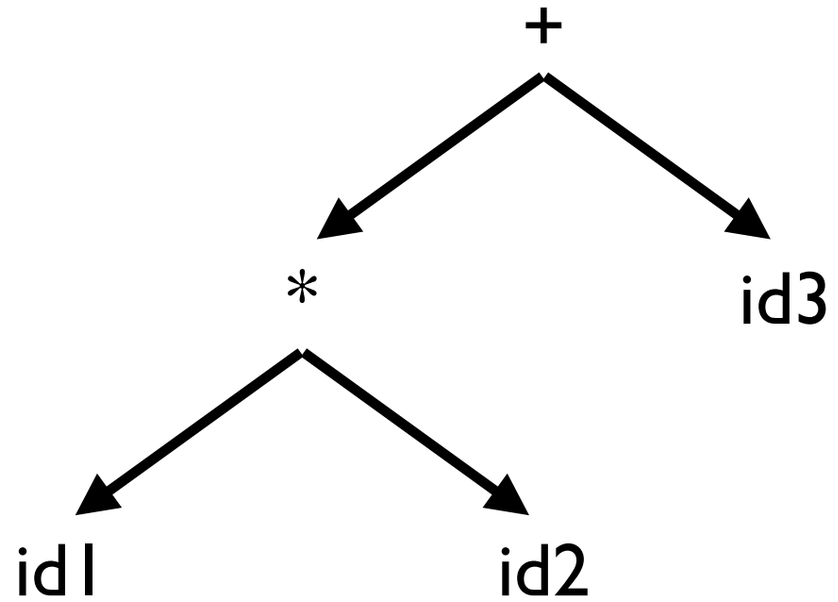


Syntax Tree

- **Parse Tree** = "Concrete" Syntax Tree
- **Syntax Tree** = "Abstract" Syntax Tree (AST)
  - Condensed form of the parse tree
  - Does not contain all information in the program
    E.g., spacing, comments, brackets, parentheses
  - Any ambiguity has been resolved
    E.g., a + b + c produces the same AST as (a + b) + c
  - Chain of single productions may be collapsed, and operators move to the parent nodes

Syntax Tree





Constructing an AST

- Each node can be represented as a record
- *operators*: one field for operator, remaining fields ptrs to operands
  `mknode(op,left,right)`
- *identifier*: one field with label id and another ptr to symbol table
  `mkleaf(id,entry)`
- *number*: one field with label num and another to keep the value of the number
  `mkleaf(num,val)`

Constructing the AST

$E := E_1 + T$	$E.ptr = mknode(+, E_1.ptr, T.ptr)$
$E := T$	$E.ptr = T.ptr$
$T := T_1 * F$	$T.ptr = mknode(*, T_1.ptr, F.ptr)$
$T := F$	$T.ptr = F.ptr$
$F := (E)$	$F.ptr = E.ptr$
$F := id$	$F.ptr = mkleaf(id, \underline{entry.id})$
$F := num$	$F.ptr = mkleaf(num, val)$